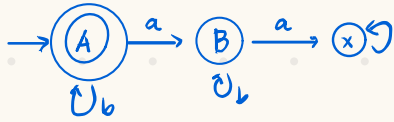


1 Language Strings over $\{a, b\}$ that do not contain the substring "aa"

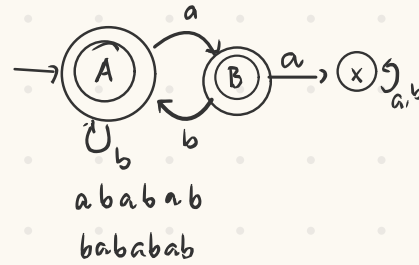
DESCRIPTION of the STATE MACHINE

State 1 is the start state and the accepting state. It transitions to state 2 on input 'a' and to itself on input 'b'. State 2 transitions to itself on input 'b' and is a trap state on input 'a'.

State machine as described



Debugged State Machine



$$Q = \{A, B, X\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = A$$

$$F = \{A, B\}$$

$$\delta: Q \times \Sigma \rightarrow Q$$

	A	B	X
a	B	X	X
b	A	A	X

My description on the debugged state machine

① First state (A) is both the initial and accepting state;

If the input is either 'a' or 'b', it will be accepted by state

Since, we want to avoid having substring "aa",

state A only transitions input if it's 'a' otherwise,

input 'b' stays in the accepting state

if input a transitions to state B → a or bba

else stays in state A → b

Possible strings bba, ba, bbba

② Second state (B) transitions input 'b' back to accepting stage

Otherwise, input 'a' is sent to state x which is a dead state and the string becomes invalid. This way of

handling is strict to prevent substring "aa"

if input 'b' transitions to state A → ab, bbab, bab

else transitions to dead state → aa x

possible initial string bbab, ab, bab

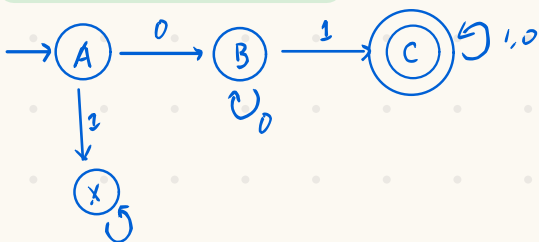
③ What will happen later on is that the transitions will loop between state A and B. And the string will always end either {a,b}. Possible strings accepted by the state machine: ✓ abababab ✓ bbababab ✓ babab

2 Language Strings over $\{0, 1\}$ that start with '0' and have at least two '1's

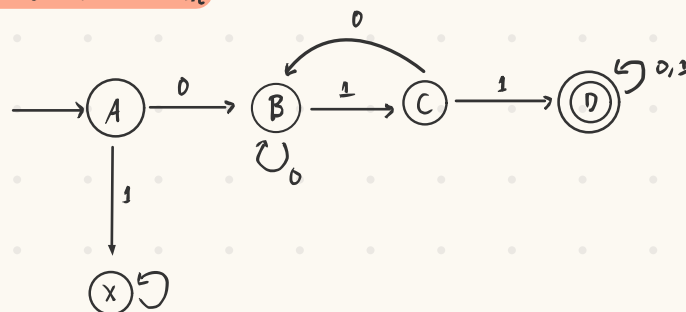
DESCRIPTION of the STATE MACHINE

State 1 is the start state. It transitions to State 2 on input '0' and is a trap state on input '1'. State 2 transitions to State 3 on input '1', and to itself on input '0'. State 3 is the accepting and transitions to itself on both inputs.

State machine as described



Debugged State Machine



$$Q = \{A, B, C, D, X\}$$

$$\delta: Q \times \Sigma \rightarrow Q$$

$$\Sigma = \{0, 1\}$$

$$q_0 = A$$

$$F = \{C\}$$

	A	B	C	D	X
0	B	B	B	D	
1	X	C	D	D	

My description on the debugged state machine

The un-debugged state machine only has three states. It does make sure that the string starts with input '0' instead of 1. It directly send to a dead state. Same thing to the debugged state it rejects input '1' unless it's '0'. The following states makes sure that there are two '1' string.